

IMAGE를 사용한 감정 분류

인공지능데이터사이언스학과 24011917 이지현

1. INTRODUCTION

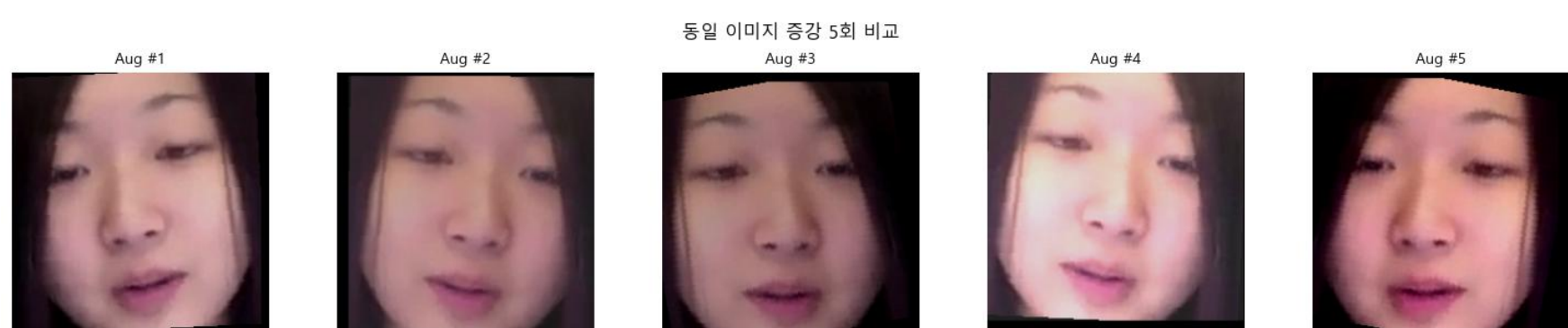
[목적] - MOSI 데이터셋의 영상 프레임에서 긍·부정 감정을 분류하는 이진 분류 모델을 구현한다. 파라미터 수가 적은 경량 모델 efficientnet 을 파인 튜닝하여 성능을 높이는 것을 목표로 한다.

[파이프라인] - transform 증강 기법을 추가해 데이터 전처리를 진행하고 efficientnet 시리즈 중 가장 가벼운 B0 모델을 이용하여 총 24개의 하이퍼파라미터 조합을 순차 실험했다. 그 중 Best 모델은 기본 CNN, AlexNet보다 더 높은 성능을 보였다.

2. METHODOLOGY

전처리: 기하학적 데이터 증강

RandomHorizontalFlip / RandomRotation($\pm 15^\circ$) / ColorJitter / RandomAffine / ImageNet 정규화 ($\mu=0.485/0.456/0.406$, $\sigma=0.229/0.224/0.225$) → 현실에서 발생 가능한 위치, 형태 변화에 대응



모델 구조: 가벼운 모델로 높은 성능 계획

ImageNet pretrained 가중치 로드 → 마지막 FC 레이어를 2-class로 교체 후 전체 fine-tuning → 사전 학습된 가중치를 활용하여 부족한 데이터셋에서도 높은 일반화 성능 확보

하이퍼파라미터 탐색: Grid Search

Optimizer : Adam / SGD
LR : $1e-3 / 1e-4 / 1e-5$
Epochs : 5 / 10 / 15 / 20
총 24가지 조합 순차 실험 → Best: Adam, lr= $1e-4$, epoch=10 → 모델의 수렴 속도와 분류 정확도를 극대화하는 최적 조합 도출

3. RESULTS

[모델별 Test Accuracy 비교]

SimpleCNN (MaxPool+Dropout) : 55.9%
SimpleCNN (AvgPool) : 60.7%
AlexNet : 57.3%
EfficientNet-B0 : 68.4% ★

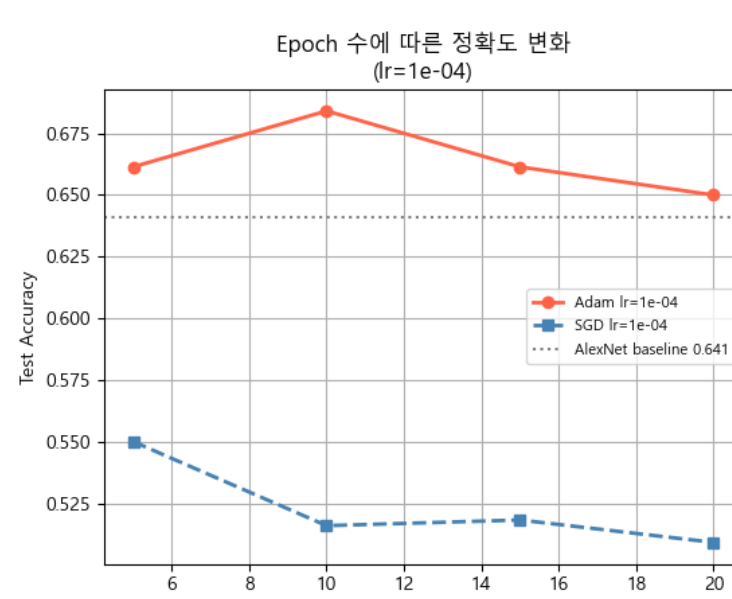
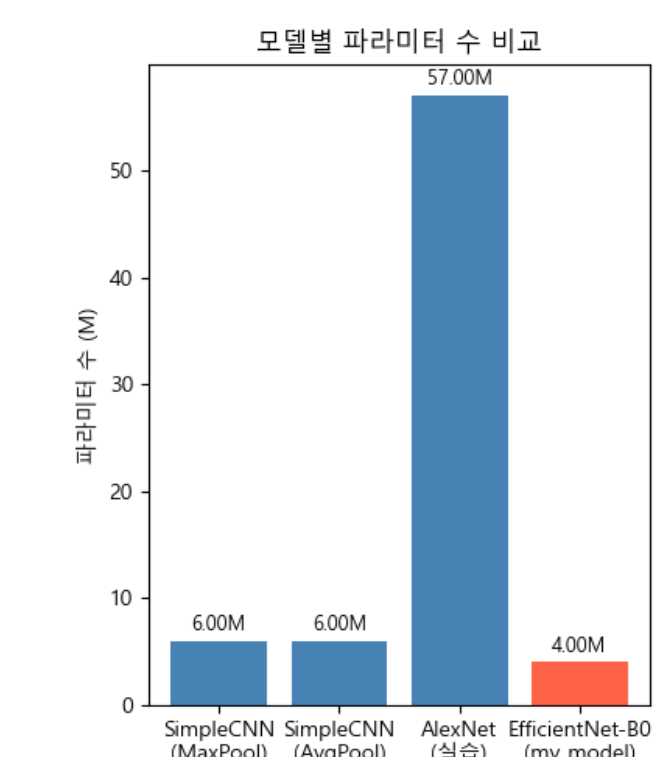
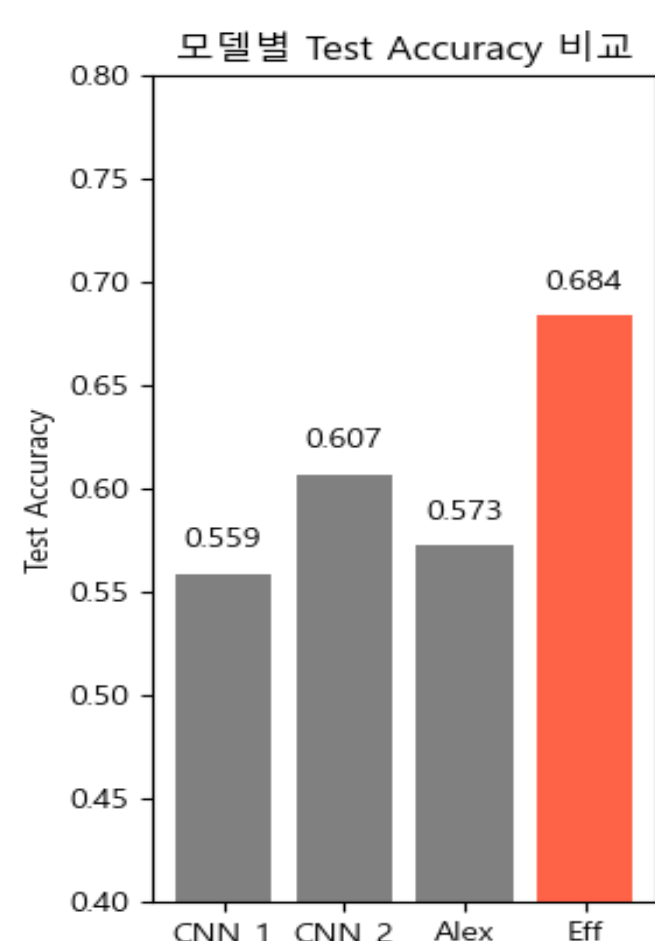
Best 모델 EfficientNet-B0은 Test Accuracy 68.41%, F1-Score 68.39% 의 결과가 나왔다. Accuracy와 F1-Score가 거의 동일한 것은 클래스 간 예측 균형이 잘 맞춰졌음을 의미한다. 약 6M개의 파라미터를 가진 SimpleCNN보다 더 가벼운 경량 모델로 Test Accuracy가 높게 나왔다는 점에 의의가 있다.

[모델별 Parameter 비교]

SimpleCNN (MaxPool+Dropout) : 6,446,498
SimpleCNN (AvgPool) : 6,446,498
AlexNet : 57,012,034
EfficientNet-B0 : 4,010,110 ★

[하이퍼파라미터 실험 요약]

- Adam이 SGD 대비 전반적으로 높은 정확도
- lr= $1e-4$ 에서 가장 안정적인 수렴
- epoch 증가에 따른 성능 향상은 10 이후 과대적합



4. CODE

Part2. 데이터 전처리 - 데이터 품질 개선 및 증강

```
def pipeline_D(path):  
    img = load_image(path)  
    if img is None:  
        return None  
    img = face_crop(img)  
    img = pad_img(img)  
    img = denoise_img(img)  
    img = sharpen_img(img)  
    return img  
  
train_transforms = transforms.Compose([  
    transforms.ToPILImage(),  
    transforms.RandomHorizontalFlip(p=0.5),  
    transforms.RandomRotation(degrees=15),  
    transforms.ColorJitter(brightness=0.3,  
                           contrast=0.3, saturation=0.2),  
    transforms.RandomAffine(degrees=0,  
                           translate=(0.05, 0.05)),  
    transforms.ToTensor(),  
    transforms.Normalize(mean=[0.485, 0.456, 0.406],  
                        std=[0.229, 0.224, 0.225]),])
```

Part3. 모델 구조 설계

```
e_model = efficientnet_b0(weights=EfficientNet_B0_Weights.IMAGENET1K_V1)  
in_features = e_model.classifier[1].in_features  
e_model.classifier[1] = nn.Linear(in_features, 2)
```

Part4. 모델 학습 - 24개의 파라미터 조합 생성

```
LR_LIST = [1e-3, 1e-4, 1e-5]  
EPOCH_LIST = [5, 10, 15, 20]  
OPTIMIZER_LIST = ['Adam', 'SGD']  
# 모든 조합 생성  
grid_configs = [  
    {'lr': lr, 'num_epochs': ep, 'optimizer': opt}  
    for lr, ep, opt in product(LR_LIST, EPOCH_LIST, OPTIMIZER_LIST)]
```

Part4. 모델 학습 - build 함수

```
def build_model():  
    #매 실험마다 동일한 pretrained 가중치로 초기화.  
    m = efficientnet_b0(weights=EfficientNet_B0_Weights.IMAGENET1K_V1)  
    in_feat = m.classifier[1].in_features  
    m.classifier[1] = nn.Linear(in_feat, 2)  
    return m.to(device)  
  
def build_optimizer(model, opt_name, lr):  
    if opt_name == 'Adam':  
        return optim.Adam(model.parameters(), lr=lr)  
    elif opt_name == 'SGD':  
        return optim.SGD(model.parameters(), lr=lr, momentum=0.9)  
    raise ValueError(f'Unknown optimizer: {opt_name}')
```

Part4. 모델 학습 - 조합 별 acc,f1, 소요 시간 등 저장/최적 모델 탐색

```
for i, cfg in enumerate(grid_configs):  
    lr = cfg['lr']  
    num_ep = cfg['num_epochs']  
    opt_name = cfg['optimizer']  
    print(f'\n[{i+1}/{len(grid_configs)}] lr={lr} epochs={num_ep} optimizer={opt_name}')  
    print('-' * 50)  
    # 모델 & 옵티마이저 초기화  
    exp_model = build_model()  
    exp_optim = build_optimizer(exp_model, opt_name, lr)  
    t0 = time.time()  
    # 학습  
    exp_train_loss, exp_train_acc = train_model(  
        exp_model, train_loader, criterion, exp_optim, num_ep, device)  
    # 평가  
    test_loss_e, test_acc_e, preds_e, labels_e = evaluate_model(  
        exp_model, test_loader, criterion, device)  
    elapsed = time.time() - t0  
    f1 = f1_score(labels_e, preds_e, average='weighted')  
    grid_results.append({  
        'optimizer': opt_name, 'lr': lr, 'num_epochs': num_ep,  
        'test_acc': round(test_acc_e, 4), 'f1_score': round(f1, 4),  
        'final_train_acc': round(exp_train_acc[-1], 4), 'elapsed_sec': round(elapsed, 1),})
```

5. CONCLUSION

EfficientNet-B0은 약 4M의 파라미터로 AlexNet(57M) 대비, 약 14배 경량이면서도 전이 학습을 통해 감정 분류 성능을 상회하였다. 적은 데이터지만 rotation, colorjitter 등을 이용해 데이터를 증강한 전처리가 모델 안정성에 기여한 것으로 판단된다. Grid Search 결과 Adam(lr= $1e-4$, epoch=10)조합이 최적으로 확인되었으며, 경량 모델로도 충분한 성능을 달성할 수 있음을 검증하였다. 그러나 정확도 70%까지는 도달하지 못하여 추후 거대 모델로 학습하여 성능을 높인 후 이를 경량화하는 역과정을 도출하고자 한다.